

Message-Passing Interface (MPI)

8

“Perhaps it doesn’t understand English”

Alice’s Adventures in Wonderland, Lewis Carroll

MPI is the state-of-the-art approach for programming supercomputers. It provides an API that was standardized by the research community over several decades,¹ and it provides a portable API for computing on distributed-memory systems—including clusters and supercomputers.

1: <https://www.mpi-forum.org/>

Notes

- ▶ For more on this topic, do consult Volume 2 of Parallel Programming for Science and Engineering².
- ▶ This section focuses on *distributed* memory, in contrast with the *shared* memory approaches we saw earlier—e.g., OpenMP, PThreads.
- ▶ Memory is distributed across nodes/servers, and MPI abstracts the network transport(s) that are in place between nodes: you *could* use TCP, but it’s not always a necessary choice (e.g., if communication is done within the same host) or the best choice (e.g., if there are higher-throughput, intra-cluster links such as InfiniBand or Omni-Path).
- ▶ There are similarities with Chapter 2—there we used distributed memory, serialisation of data types, and abstracting communication channels. Those show up in MPI too. Our exploration of the MPI API builds on the discussion about API design decisions from Chapter 2.
- ▶ Another important consideration MPI is *portability* across platforms. This matters since it makes code resilient to different underlying platform and network hardware. See the hwloc tool that provides portable hardware locality information³ Examples of its outputs are shown in Fig 8.1 and Fig 8.2.
- ▶ The paradigm we use in MPI is sometimes called SPMD—for “Single Program, Multiple Data”. Usually, a *single* program is distributed across a distributed memory infrastructure, but the program’s behavior on each node differs depending on the data that it’s provided, and the communication patterns that arise from the program’s communicator and *rank*—a rank is a unique identifier for a program within the set of running programs.
- ▶ There are two open-source implementations of the MPI standard—MPICH⁴ and Open MPI. There are also vendor-specific implementations—4: See <https://www.mpich.org/> such as those by Microsoft, Intel, Cray/HPE, IBM, etc. What we learn in this course should work with any implementation that complies with the MPI standard.
- ▶ Among other tools, MPI provides tools to compile and to execute programs across a pool of resources.
- ▶ We have MPICH set up across several nodes in a FABRIC slice.
- ▶ You can use MPI with arbitrary programs. For instance:

```
ubuntu@n1:~$ echo "Hello, world"
```

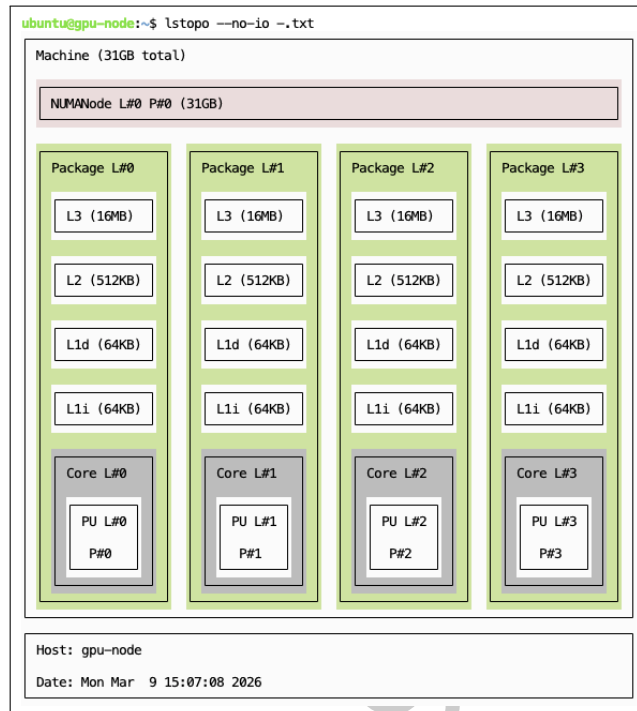


Figure 8.1: Visualization provided by Open MPI's tooling based on its resource-discovery tool. This shows the number of NUMA nodes and their structure. In particular, we can see the structure and size of caches.

Hello, world

```
ubuntu@n1:~$ mpiexec -n 4 echo "Hello, world"
```

Hello, world

Hello, world

Hello, world

Hello, world

and

```
ubuntu@n1:~$ mpiexec -f hosts echo "Hello, world"
```

Hello, world

Hello, world

Hello, world

Hello, world

Hello, world

Hello, world

Hello, world

Hello, world

where 'hosts' contains:

```
n1:2
```

```
n2:2
```

```
n3:2
```

```
n4:2
```

where those hosts have the following addresses:

```
ubuntu@n1:~$ host -s n2
```

```
n2 has address 10.0.2.2
```

```
Host n2 not found: 3(NXDOMAIN)
```

```
ubuntu@n1:~$ host -s n3
n3 has address 10.0.3.2
Host n3 not found: 3(NXDOMAIN)
ubuntu@n1:~$ host -s n4
n4 has address 10.0.4.2
Host n4 not found: 3(NXDOMAIN)
```

- What if we run a program that returns different outputs on different hosts?

```
ubuntu@n1:~$ hostname
n1
ubuntu@n1:~$ mpiexec -f hosts hostname
n1
n1
n2
n2
n3
n3
n4
n4
```

- We can direct mpiexec to start a given number of instances on the current same host:

```
ubuntu@n1:~$ mpiexec -n 6 hostname
n1
n1
n1
n1
n1
n1
```

- You can use MPI with programs that use the MPI API. We'll look at this next:

```
1 #include "mpi.h"
2 #include "stdio.h"
3
4 int
5 main (int argc, char **argv)
6 {
7     MPI_Init(&argc, &argv);
8
9     int size;
10
11     MPI_Comm_size(MPI_COMM_WORLD, &size);
12
13     printf("Size is %d\n", size);
14
15     MPI_Finalize();
16 }
```

MPI's API

Look up the API's functions through *manual pages*.

```
ubuntu@n1:~$ mpicc mpi1.c
ubuntu@n1:~$ mpiexec -n 5 ./a.out
Size is 5
Size is 5
```

```
Size is 5
Size is 5
Size is 5
```

► We evolve our example further:

```
1 #include "mpi.h"
2 #include "stdio.h"
3
4 int
5 main (int argc, char **argv)
6 {
7     MPI_Init(&argc, &argv);
8
9     int size;
10    int rank;
11
12    MPI_Comm_size(MPI_COMM_WORLD, &size);
13    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
14
15    printf("From rank %d: Size is %d\n", rank, size);
16
17    MPI_Finalize();
18 }
```

Parameter-passing

When reviewing the MPI example code, recall pass-by-value vs pass-by-reference.

5: |mpicc| is *not* a compiler—it’s a wrapper that invokes a compiler.

Next we compile the code, and we run it. “mpicc” will call our regular C compiler but provide it with additional parameters for finding MPI inclusions and libraries.⁵ The “-n 4” parameter to “mpiexec” states that we wish to start 4 processes.

```
ubuntu@n1:~$ mpicc mpi1.c
ubuntu@n1:~$ mpiexec -n 5 ./a.out
From rank 0: Size is 5
From rank 2: Size is 5
From rank 4: Size is 5
From rank 1: Size is 5
From rank 3: Size is 5
```

Parameter-passing

When reviewing the MPI example code, recall pass-by-value vs pass-by-reference.

- There is a *lot* more that can be configured when executing MPI programs—including process affinity, for example. For more details, see the manual page for “mpiexec”.
- Every process starts up within the MPI context. That context includes a *communicator*, or communication domain. In the previous example, the communicator consists of MPI_COMM_WORLD—this is a special value that refers to the entire set of MPI processes. (MPI processes can be structured into topologies to narrow down the possible communication paths that processes can use to communicate with each other.) A process receives its “identity” (rank) from the MPI environment.
- Next we see how to specialise the behaviour of processes according to their *rank*:

```
1 #include "mpi.h"
2 #include "stdio.h"
3
4 int
5 main (int argc, char **argv)
6 {
7     MPI_Init(&argc, &argv);
```

```

8
9  int size;
10 int rank;
11
12 MPI_Comm_size(MPI_COMM_WORLD, &size);
13 MPI_Comm_rank(MPI_COMM_WORLD, &rank);
14
15 if (0 == rank)
16     printf("From rank %d: Size is %d\n", rank, size);
17
18 MPI_Finalize();
19 }

```

```

ubuntu@n1:~$ mpicc mpi1.c
ubuntu@n1:~$ mpiexec -n 5 ./a.out
From rank 0: Size is 5
ubuntu@n1:~$ ./a.out
From rank 0: Size is 1

```

- Each MPI process can exchange data with other processes. Each process receives a unique identifier from the MPI runtime, and that identifier is used to send/receive data to/from a specific process. Check out “`man MPI_Send`” and “`man MPI_Recv`”. to find out about those functions.
- Let’s start with the following declaration:

```

1 int MPI_Send(void *buf, int count, MPI_Datatype datatype,
    int dest, int tag, MPI_Comm comm);

```

Note examples of MPI_Datatype: MPI_CHAR, MPI_SHORT, MPI_INT, MPI_UNSIGNED, MPI_FLOAT, etc. These serve to abstract the representations of data—which is a topic that comes up in systems programming.⁶

6: For example, see <http://www.cs.iit.edu/~nsultana1/teaching/F22CS351/>

- Note the following:
 - Data transfer is strictly typed—in other words, the receiver can only receive the type of data that the MPI_Recv function has been instructed to expect.
 - Data can be further “tagged” by a user-defined, int-valued label. This can simply be set to 0.
 - The MPI_Recv allows generic behaviour with regards to source and tag, through the “MPI_ANY_SOURCE” “MPI_ANY_TAG” wildcards.
 - The MPI_Recv call populates a “MPI_Status” value, from whose fields we can obtain useful information: MPI_SOURCE and MPI_TAG.
 - Moreover, we can use MPI_Get_count to get the size of the received data.
- Let’s write our first example that involves sending and receiving data:

```

1 #include <mpi.h>
2 #include <stdio.h>
3
4 int
5 main (int argc, char **argv) {
6     int size;
7     int rank;
8
9     MPI_Init(&argc, &argv);

```

```

10
11 MPI_Comm_size(MPI_COMM_WORLD, &size);
12 MPI_Comm_rank(MPI_COMM_WORLD, &rank);
13
14 printf("Started: rank %d out of %d\n", rank, size);
15
16 if (0 == rank % 2) {
17     int data = 5;
18     MPI_Send(&data, 1, MPI_INT, (rank + 1) % 2, 0,
19             MPI_COMM_WORLD);
20     printf("Rank %d sent %d\n", rank, data);
21 } else {
22     int result = -1;
23     MPI_Status status;
24     MPI_Recv(&result, 1, MPI_INT, //(rank + 1) % 2
25             MPI_ANY_SOURCE, MPI_ANY_TAG, MPI_COMM_WORLD, &
26             status);
27     printf("Rank %d received %d\n", rank, result);
28 }
29
30 MPI_Finalize();
31
32 return 0;
33 }

```

```

ubuntu@n1:~/test$ mpiexec -n 2 ./a.out
Started: rank 0 out of 2
Started: rank 1 out of 2
Rank 0 sent 5
Rank 1 received 5

```

Then:

```

ubuntu@n1:~/test$ mpiexec -n 3 ./a.out
Started: rank 0 out of 3
Started: rank 1 out of 3
Started: rank 2 out of 3
Rank 0 sent 5
Rank 1 received 5
Rank 2 sent 5

```

But then:

```

ubuntu@n1:~/test$ mpiexec -n 4 ./a.out
Started: rank 0 out of 4
Rank 0 sent 5
Started: rank 1 out of 4
Rank 1 received 5
Started: rank 2 out of 4
Rank 2 sent 5
Started: rank 3 out of 4
^C[mpiexec@n1] Sending Ctrl-C to processes as requested
[mpiexec@n1] Press Ctrl-C again to force abort

```

```

=====
= BAD TERMINATION OF ONE OF YOUR APPLICATION PROCESSES
= PID 161602 RUNNING AT n1

```

```
= EXIT CODE: 2
= CLEANING UP REMAINING PROCESSES
= YOU CAN IGNORE THE BELOW CLEANUP MESSAGES
```

```
=====
YOUR APPLICATION TERMINATED WITH THE EXIT STRING: Interrupt (signal 2)
This typically refers to a problem with your application.
Please see the FAQ page for debugging suggestions
```

```
1 #include <mpi.h>
2 #include <stdio.h>
3
4 int
5 main (int argc, char **argv) {
6     int size;
7     int rank;
8
9     MPI_Init(&argc, &argv);
10
11     MPI_Comm_size(MPI_COMM_WORLD, &size);
12     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
13
14     printf("Started: rank %d out of %d\n", rank, size);
15
16     if (0 == rank % 2) {
17         for (int data = 5; data > 0; data--) {
18             MPI_Send(&data, 1, MPI_INT, (rank + 1) % 2, 0,
19                 MPI_COMM_WORLD);
20             printf("Rank %d sent %d\n", rank, data);
21         }
22     } else {
23         int result = -1;
24         do {
25             MPI_Status status;
26             MPI_Recv(&result, 1, MPI_INT, //(rank + 1) % 2
27                 MPI_ANY_SOURCE, MPI_ANY_TAG, MPI_COMM_WORLD, &
28                 status);
29             printf("Rank %d received %d\n", rank, result);
30         } while (result > 0);
31     }
32
33     printf("Ending: rank %d\n", rank);
34
35     MPI_Finalize();
36
37     return 0;
38 }
```

```
ubuntu@n1:~/test$ mpiexec -n 2 ./a.out
Started: rank 0 out of 2
Rank 0 sent 5
Rank 0 sent 4
Rank 0 sent 3
Rank 0 sent 2
Rank 0 sent 1
Ending: rank 0
Started: rank 1 out of 2
Rank 1 received 5
```

```
Rank 1 received 4
Rank 1 received 3
Rank 1 received 2
Rank 1 received 1
```

The process seems to hang. Pressing Ctrl-C gets us nowhere:

```
^C[mpiexec@n1] Sending Ctrl-C to processes as requested
[mpiexec@n1] Press Ctrl-C again to force abort
```

```
=====
= BAD TERMINATION OF ONE OF YOUR APPLICATION PROCESSES
= PID 161527 RUNNING AT n1
= EXIT CODE: 2
= CLEANING UP REMAINING PROCESSES
= YOU CAN IGNORE THE BELOW CLEANUP MESSAGES
=====
```

```
YOUR APPLICATION TERMINATED WITH THE EXIT STRING: Interrupt (signal 2)
This typically refers to a problem with your application.
Please see the FAQ page for debugging suggestions
```

- After fixing that off-by-one bug, we get the expected output:

```
ubuntu@n1:~/test$ mpicc mpi4.c
ubuntu@n1:~/test$ mpiexec -n 2 ./a.out
Started: rank 0 out of 2
Rank 0 sent 5
Rank 0 sent 4
Rank 0 sent 3
Rank 0 sent 2
Rank 0 sent 1
Rank 0 sent 0
Ending: rank 0
Started: rank 1 out of 2
Rank 1 received 5
Rank 1 received 4
Rank 1 received 3
Rank 1 received 2
Rank 1 received 1
Rank 1 received 0
Ending: rank 1
```

- A simple example that we saw in class:

```
1 #include "assert.h"
2 #include "mpi.h"
3 #include "stdio.h"
4 #include "stdlib.h"
5 #include "time.h"
6 #include "unistd.h"
7
8 #define SENDER 0
9 #define RECEIVER 1
10
11 int
12 main (int argc, char **argv)
13 {
```



```

14  srand(time(NULL));
15  MPI_Init(&argc, &argv);
16
17  int size;
18  int rank;
19
20  MPI_Comm_size(MPI_COMM_WORLD, &size);
21  MPI_Comm_rank(MPI_COMM_WORLD, &rank);
22
23  assert(2 == size);
24
25  if (SENDER == rank)
26      printf("Started: from rank %d: Size is %d\n", rank,
27             size);
28
29  switch (rank) {
30      case SENDER:
31          int data = rand() % 10;
32          printf("SENDER: %d\n", data);
33          /*FIXME check return value*/MPI_Send(&data, 1, MPI_INT,
34          RECEIVER, 0, MPI_COMM_WORLD);
35          break;
36      case RECEIVER:
37          {
38              int data = -1;
39              MPI_Status status;
40              /*FIXME check return value*/MPI_Recv(&data, 1,
41              MPI_INT, MPI_ANY_SOURCE, MPI_ANY_TAG,
42              MPI_COMM_WORLD, &status);
43              printf("RECEIVER: %d from %d\n", data, status.
44              MPI_SOURCE);
45          }
46          break;
47      default: assert(0);
48  }
49
50  MPI_Finalize();
51 }

```

► Examples we could write:

- Sending a value through a pipeline of processes.
- Having processes each produce a random value, and then finding which of them has the maximum value.
- Dining philosophers.

► Note that communication can be

- *point-to-point*: one process sends data to another process.
- *collective*: more than two processes involved.

► Examples of collectives include: broadcast (`MPI_Bcast`), scatter and gather (e.g., `MPI_Gather`), and `MPI_Reduce` (possibly partial computation over data being communicated).

MPI or not MPI?

Based on what you know so far, how would you go about thinking whether to use MPI? For what examples of tasks do you think MPI would be *unsuitable*?

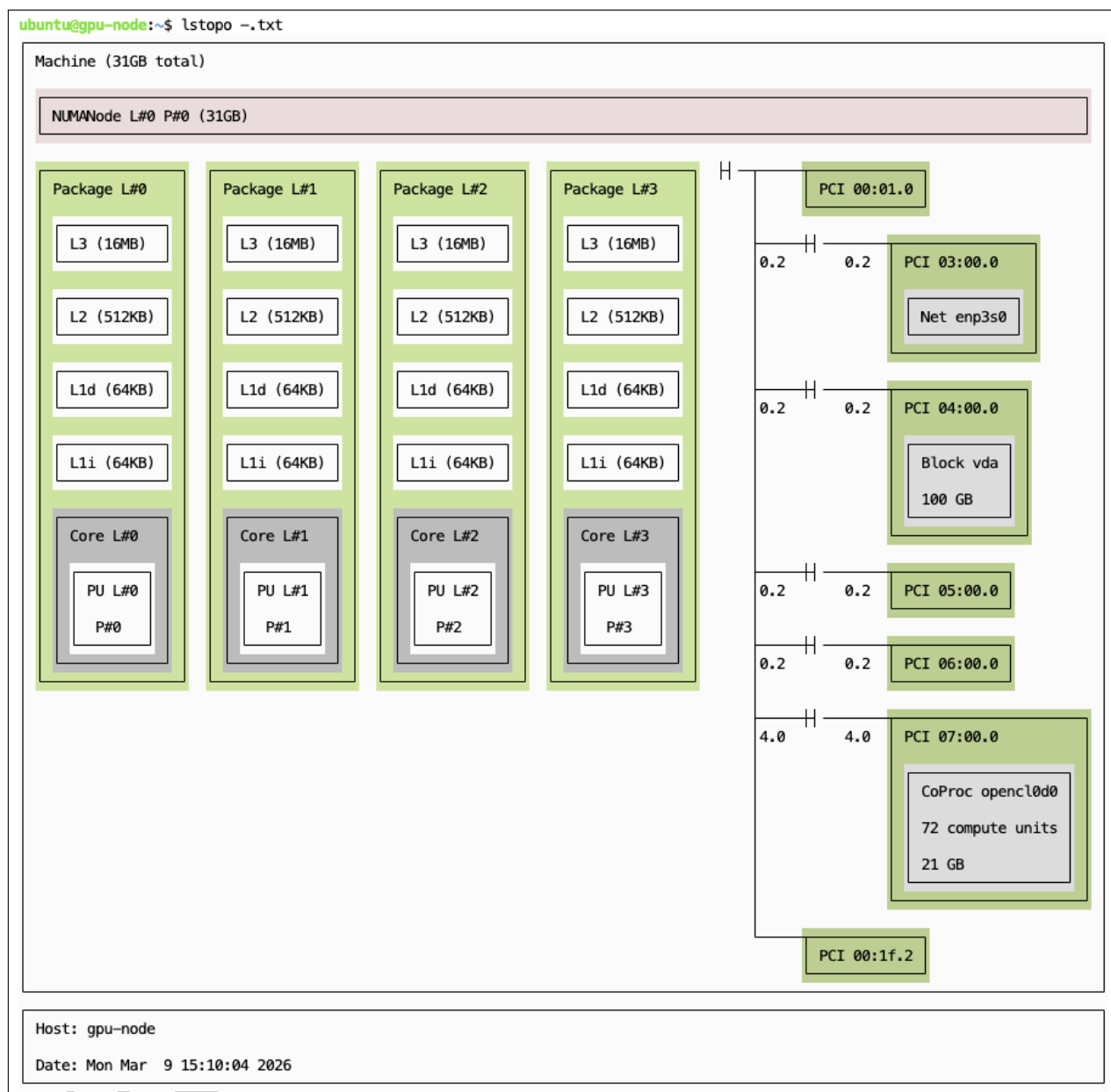


Figure 8.2: In addition to processor resources, the tool referenced in Fig. 8.1 can also enumerate and visualize other system resources.